

ESLtopia — Test Case Specification

Detailed test cases mapped to the automated Playwright suite · v1.7 · 2026-07-06

Scope	Every automated test currently defined in <code>tests/</code> (15 spec files, 144 defined test cases)
Companion documents	Master Test Plan · Test Scenario Coverage Checklist · docs/qa/test-coverage-todo.md
Legend	UI browser-driven API direct REST/Auth call SKIPPED gated behind an undeployed feature flag

v1.7 revision: No new cases and no case count change (still 144 across 15 spec files). A live CI verification run found that three of the modules brought into cross-browser execution by v1.6 send real Supabase emails (Module 3 Registration, Module 4 Forgot Password, Module 9 School → Teachers Management) — running them under three engines in one run exhausted a shared email rate-limit bucket and failed 5 SCHAT cases on WebKit. Those three modules are now Chromium-only for cross-browser purposes too, alongside Modules 10/11/14 (pure-REST/visual). Re-verified via a second live CI run: 304 passed, 0 failed. See Test Plan §11/§13 for the incident and the exclusion list this document doesn't track per-ID.

v1.6 revision: No new cases and no case count change (still 144 across 15 spec files) — this revision documents that most of the suite now also executes under Firefox and WebKit (two new `playwright.config.ts` projects), not just Chromium. `classes.spec.ts`, `profile.spec.ts`, and the Visual Regression module (§14) are Chromium-only by design; every other module's cases are the same IDs, just now run up to three times each. See Test Plan §4.1/§13 for the browser-execution details this document doesn't track per-ID.

v1.5 revision: Closed three high-priority items from `test-coverage-todo.md`. Added Module 15, **Classes Tab — UI** (CLSUI-01–06, new file `tests/app/classes-ui.spec.ts`), covering the "add class" form, roster add/remove inputs, the delete-class button, and XSS/HTML-injection resilience in class and student names. Added **RECU-09** (Records Tab — UI §12) for the same XSS check on the notes field. Added **LOGIN-17** and **FPW-08** exercising a small burst of repeated failed attempts against `/auth/v1/token` and `/auth/v1/recover` respectively — both document current production behavior (no logout or 429 observed within 4–5 attempts on either endpoint as of 2026-07-06) rather than asserting a specific rate-limit threshold, since none is currently enforced; see the Test Plan §11 risk note. Case count: 135 → 144; spec files: 14 → 15.

v1.4 revision: Added the visual-regression suite as Module 14 (VIS-01–04, `tests/visual/visual-regression.spec.ts`) — it existed in the repo (merged via the `visual-regression-tests` branch) but was never added to this spec. Also added two cases that were already implemented in code but missing from this document: **REG-11** (Registration §3 — duplicate-email signup returns 200 with empty identities, an anti-enumeration check) and **CLS-08** (Classes & Records §10 — RLS blocks inserting a class row with another teacher's forged `owner_id`). Inserting each in file order shifted every later ID in its module up by one (old REG-11/12 → REG-12/13; old CLS-08–15 → CLS-09–16). Case count: 129 → 135; spec files: 13 → 14.

v1.3 revision: Added RECU-08 to close the record-sync failure gap flagged in the Test Scenario Coverage Checklist §7 — `syncRecord` logs a failed upsert instead of discarding it silently, but nothing verified that path fired, or that logging doesn't mask the write still being lost. Case count: 128 → 129. Both scenario gaps the v1.1 review surfaced are now closed.

v1.2 revision: Added SCHAT-14 and SCHAT-15 to close the teacher-removal cascade gap flagged in the Test Scenario Coverage Checklist §5 (and in this doc's own v1.1 note) — the "remove" happy-path test previously gave the teacher no classes/records to lose, so the cascade delete (`classes.owner_id` / `records.owner_id` both cascade from `auth.users`) was never actually verified. Case count: 126 → 128.

v1.1 revision: A full code review pass (app code + all tests + edge functions + migrations + config) fixed 8 issues, none of which changed test-case counts or expected results — School → Teachers Management and Superadmin — Admin Panel &

RLS's `invoke / invokeEdgeFn` helpers were deduplicated into `tests/helpers/edgeFunctions.ts` (internal-only change, same assertions), and a stale comment in `superadmin.spec.ts` describing pre-migration DELETE-policy behavior was corrected to match what SADM-12 through SADM-15 already verified. All 126 cases were re-run against the fixed build and pass.

16 cases in School → Teachers Management and 2 cases in Superadmin RLS: manage-teacher gate are currently **SKIPPED** — the feature is code-complete but not yet deployed (DB migration + edge functions + app). They run automatically once `FEATURE_SCHOOL_TEACHERS=1` is set after deploy — which is the case in CI today.

Module Summary

#	Module	Spec file	Cases
1	Login	<code>tests/auth/login.spec.ts</code>	17
2	Logout	<code>tests/auth/logout.spec.ts</code>	5
3	Registration	<code>tests/auth/registration.spec.ts</code>	13
4	Forgot Password	<code>tests/auth/forgot-password.spec.ts</code>	8
5	Password Reset	<code>tests/auth/password-reset.spec.ts</code>	5
6	Session & Navigation Guards	<code>tests/auth/navigation.spec.ts</code>	3
7	Roles & Permissions	<code>tests/auth/roles.spec.ts</code>	13
8	Superadmin — Admin Panel & RLS	<code>tests/auth/superadmin.spec.ts</code>	19
9	School → Teachers Management	<code>tests/auth/school-teachers.spec.ts</code>	16
10	Classes & Records — Data Layer (REST)	<code>tests/app/classes.spec.ts</code>	16
11	Profile Self-Management	<code>tests/app/profile.spec.ts</code>	5
12	Records Tab — UI	<code>tests/app/records.spec.ts</code>	9
13	PDF Preview Modal	<code>tests/app/pdf-modal.spec.ts</code>	5
14	Visual Regression	<code>tests/visual/visual-regression.spec.ts</code>	4
15	Classes Tab — UI	<code>tests/app/classes-ui.spec.ts</code>	6
Total			144

1. Login

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
LOGIN-01	UI	High	Valid credentials → logged in	Confirmed teacher fixture exists	1. Go to / 2. Fill email + password 3. Submit	"Get started" button visible within 15s
LOGIN-02	UI	High	Wrong password → generic error	Fixture exists	Submit with correct email, wrong password	"Incorrect email or password." shown; sign-in form remains
LOGIN-03	UI	Med	Unregistered email → generic error	None	Submit with a never-used @example.test address	Same generic "Incorrect email or password." (no enumeration)
LOGIN-04	UI	Low	Invalid email format → client validation	None	Submit with <code>notanemail</code>	"Please enter a valid email address." shown; no network call made

LOGIN-05	UI	Low	Empty password → validation error	None	Submit with email only	"Please enter your password." shown
LOGIN-06	UI	Low	Empty email → validation error	None	Submit with password only	"Please enter a valid email address." shown
LOGIN-07	API	High	Password grant → 200 with tokens	Fixture exists	POST /auth/v1/token? grant_type=password	200; access_token, refresh_token present; token_type=bearer; expires_in>0
LOGIN-08	API	Med	Response includes confirmed user object	Fixture exists	Same POST as LOGIN-07	user.email matches; user.id is a UUID; email_confirmed_at is set
LOGIN-09	API	Med	Access token is a well-formed JWT	Fixture exists	Inspect access_token shape	Matches header.payload.signature pattern
LOGIN-10	API	High	Wrong password → 400	Fixture exists	POST with bad password	HTTP 400
LOGIN-11	API	Med	Unregistered email → 400	None	POST with unused address	HTTP 400
LOGIN-12	API	High	Missing apikey header → 401	Fixture exists	POST without apikey	HTTP 401
LOGIN-13	API	Med	Refresh token grant → new token pair	Fixture exists; prior login	POST grant_type=refresh_token with a valid refresh_token	200; new access_token + refresh_token returned
LOGIN-14	API	Med	Invalid refresh token → 400	None	POST with a bogus string	HTTP 400
LOGIN-15	API	High	Refresh token invalidated after logout	Fixture exists	1. Login 2. Logout with access token 3. Retry refresh grant with the old refresh token	HTTP 400 — refresh token no longer honored
LOGIN-16	UI	High	Unconfirmed user cannot log in	None (signup done in-test)	1. Sign up via public API, no confirmation 2. Attempt login via UI	Generic "Incorrect email or password." — no distinct "unconfirmed" message leaked
LOGIN-17	API	High	A burst of wrong-password attempts is always rejected, never a 5xx or a silent 200	Fixture exists	POST the password grant 5 times in quick succession with a different wrong password each time	Every response is neither 200 nor ≥500 (as of 2026-07-06: consistently 400 — no lockout or 429 observed within this burst size on production)

2. Logout

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
LOGOUT-01	UI	High	Sign out returns to login screen	Logged-in session	Click "Sign out"	Sign-in form + email placeholder visible within 10s
LOGOUT-02	UI	High	Sign out clears the session	Logged-in session	1. Sign out 2. Reload the page	Still shows sign-in form; no "Sign out" button (no auto-relogin from a stale session)
LOGOUT-03	API	Med	Valid token → 204	Valid access token	POST /auth/v1/logout	HTTP 204 No Content
LOGOUT-04	API	High	Revoked token rejected afterward	Valid access token	1. Logout 2. GET /auth/v1/user with the same token	HTTP 403 (revoked, vs 401 for missing auth)

LOGOUT-05	API	Low	No Authorization header → 401	None	POST logout with no auth header	HTTP 401
-----------	-----	-----	-------------------------------	------	---------------------------------	----------

3. Registration

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
REG-01	UI	High	Signup → confirm → logged in (teacher)	None	1. Sign up, select "Teacher" 2. Confirm via admin API (simulates email link) 3. Log in	"Sign out" button visible — fully authenticated
REG-02	UI	High	Signup → confirm → logged in (school)	None	Same as REG-01, role "School"	"Sign out" button visible
REG-03	UI	Low	Mismatched passwords → validation error	None	Fill form with differing password/confirm	"Passwords do not match."
REG-04	UI	Low	Short first name → validation error	None	First name = "A"	"First name must be at least 2 characters."
REG-05	UI	Low	Short last name → validation error	None	Last name = "B"	"Last name must be at least 2 characters."
REG-06	UI	Med	No role selected → validation error	None	Submit without choosing Teacher/School	"Please select a role."
REG-07	API	High	Valid signup → 200, unconfirmed user	None	POST /auth/v1/signup	200; returned user's email matches; email_confirmed_at falsy
REG-08	API	High	Trigger sets teacher profile role	None	Signup with role: teacher	profiles.role = 'teacher' after the on_auth_user_created trigger fires
REG-09	API	High	Trigger sets school profile role	None	Signup with role: school	profiles.role = 'school'
REG-10	API	High	Superadmin role in metadata is coerced	None	Signup with role: superadmin in metadata	Resulting profile role is teacher, never superadmin
REG-11	API	Med	Duplicate (already-registered) email → 200, anti-enumeration	A pre-existing confirmed user with the target email	POST /auth/v1/signup with an already-registered email	HTTP 200 with an empty identities array (Supabase's anti-enumeration response shape) — indistinguishable at the HTTP-status level from a fresh signup, so a caller can't detect that the email is taken
REG-12	API	Med	Weak password → 422	None	Signup with a 3-character password	HTTP 422
REG-13	API	Med	Missing apikey header → 401	None	POST signup without apikey	HTTP 401

4. Forgot Password

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
----	------	------	-------	---------------	-------	-----------------

FPW-01	UI	High	Valid email → confirmation shown	On forgot-password screen	Fill email, click "Send reset link"	"Check your email" + the entered address shown; "Back to sign in" visible
FPW-02	UI	Low	Invalid email format → validation error	On forgot-password screen	Fill notanemail	"Please enter a valid email address."; form stays put, no request sent
FPW-03	UI	Low	"Back to sign in" returns to login	On forgot-password screen	Click "Back to sign in"	Sign-in form with welcome text shown
FPW-04	API	High	Non-existent email → 200	None	POST /auth/v1/recover with an unused address	HTTP 200 (anti-enumeration)
FPW-05	API	High	Registered email → same 200	None	POST recover with any address	HTTP 200 — indistinguishable from FPW-04
FPW-06	API	Med	Missing email field → 4xx	None	POST recover with empty body	HTTP 400–499
FPW-07	API	Med	Missing apikey header → 401	None	POST recover without apikey	HTTP 401
FPW-08	API	High	A burst of recovery requests to the same address is accepted or throttled, never a 5xx	None	POST /auth/v1/recover 4 times in quick succession with the same address	Every response is <500 and either 200 or 429 (as of 2026-07-06: consistently 200 — no per-email cooldown observed within this burst size on production)

5. Password Reset

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
PWR-01	UI	High	Set new password → old stops working, new works	Fixture user; recovery link generated via admin API	1. Open recovery link 2. Set matching new password twice 3. Try old password 4. Try new password	"Password updated!" then signed out; old password rejected; new password logs in
PWR-02	UI	Low	Mismatched passwords → validation error	On reset screen via recovery link	Fill differing password/confirm	"Passwords do not match."; reset form remains
PWR-03	UI	Low	Weak password → validation error	On reset screen	Fill "weak" in both fields	"Password must be at least 8 characters."
PWR-04	UI	Low	Empty new password → validation error	On reset screen	Submit with both fields empty	"Password must be at least 8 characters."; form remains
PWR-05	UI	High	Recovery link is single-use	Fixture user; recovery link generated	1. Complete reset with the link 2. Revisit the same link	Second visit does not reopen the reset form; sign-in screen shown instead

6. Session & Navigation Guards

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
----	------	------	-------	---------------	-------	-----------------

NAV-01	UI	High	Unauthenticated visit shows login, not the app	No session	Visit / with no cookies/localStorage session	Sign-in form shown; no "Sign out" button present
NAV-02	UI	High	Session persists across reload	Logged-in session	Reload the page	"Get started" landing shown (session restored from localStorage); sign-in form not shown
NAV-03	UI	High	Deactivated account is signed out on reload	Logged-in session; account then set to inactive server-side	1. Log in 2. Deactivate the account via admin API while session is live 3. Reload	App detects status=inactive on the refetched profile, calls signOut(), shows sign-in form

7. Roles & Permissions

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
ROLE-01	UI	High	Signup form offers only Teacher/School	None	Open the signup form	Exactly 2 role cards ("Teacher", "School"); no "Super Admin" text or card anywhere
ROLE-02	API	High	Superadmin signup attempt is coerced	None	POST signup with role: superadmin in metadata	Resulting profile role is teacher
ROLE-03	UI	High	Teacher cannot see/open the Admin portal	Teacher fixture	Log in as teacher, inspect tab bar	No "Admin" tab; "Classes" tab visible
ROLE-04	UI	High	Pending account cannot log in	Fixture set to status=pending	Attempt login via UI	"Your account is pending. Check your email to set your password." shown; sign-in form remains
ROLE-05	UI	High	Inactive account cannot log in	Fixture set to status=inactive	Attempt login via UI	"Your account has been deactivated. Contact your school administrator." shown
ROLE-06	API	Med	Access token contains expected claims	Teacher fixture	Decode access_token payload	email, UUID sub, aud=authenticated, role=authenticated all present
ROLE-07	API	Med	user_metadata.role reflects signup role	Teacher fixture	Decode token payload	user_metadata.role = 'teacher'
ROLE-08	API	Low	Token expiry is in the future	Teacher fixture	Decode token payload	exp > current Unix time
ROLE-09	API	Med	GET /auth/v1/user matches token subject	Teacher fixture	Login, then GET /auth/v1/user	user.id equals token's sub; email matches
ROLE-10	API	Med	Malformed token → 403	None	GET /auth/v1/user with Bearer not.a.token	HTTP 403
ROLE-11	API	High	Teacher cannot self-promote to superadmin	Teacher fixture	PATCH own profile's role to superadmin via REST with own token	Role remains teacher after the attempt (DB trigger blocks it)
ROLE-12	API	High	School cannot self-promote to superadmin	School fixture	Same as ROLE-11 with a school account	Role remains school

ROLE-13	API	Med	School JWT payload has role: school	School fixture	Decode token payload	user_metadata.role='school', email, aud=authenticated correct
---------	-----	-----	-------------------------------------	----------------	----------------------	---

8. Superadmin — Admin Panel & RLS

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
SADM-01	UI	High	Superadmin sees the Admin tab	Superadmin fixture	Log in, inspect tab bar	"Admin" tab visible
SADM-02	UI	Med	Admin panel lists Delete buttons for other users	Superadmin + a target user fixture	Open Admin tab	At least one visible "Delete" button
SADM-03	UI	Med	No Delete button on superadmin's own row	Superadmin fixture	Open Admin tab, find own ("You") row	Own row has no "Delete" button
SADM-04	API	High	Own profile returns role: superadmin	Superadmin fixture	GET own profile via REST	Single row; role='superadmin'
SADM-05	API	High	Superadmin can list all profiles	Superadmin + peer fixture	GET /rest/v1/profiles with admin token	≥2 rows returned (full-table scan)
SADM-06	API	High	Teacher listing profiles sees only own row	Teacher fixture	GET /rest/v1/profiles with teacher token	Exactly 1 row, matching own email
SADM-07	API	High	Superadmin can read any profile	Superadmin + target fixture	GET a specific profile by id with admin token	1 row; correct email returned
SADM-08	API	High	Teacher cannot read another profile (RLS)	Teacher + target fixture	GET target's profile by id with teacher token	0 rows returned (RLS-filtered)
SADM-09	API	High	Superadmin can change another user's role	Superadmin + target fixture	PATCH target's role to school	HTTP 200; profile role becomes school
SADM-10	API	High	Teacher cannot change another user's role (RLS)	Teacher + target fixture	PATCH target's role with teacher token	Target's role unchanged, regardless of HTTP status
SADM-11	API	High	Superadmin can promote a user to superadmin	Superadmin + target fixture	PATCH target's role to superadmin	HTTP 200; role becomes superadmin
SADM-12	API	Med	Superadmin can delete another profile	Superadmin + target fixture	DELETE target's profile row	Profile no longer exists
SADM-13	API	Med	Superadmin cannot delete their own profile	Superadmin fixture	DELETE own profile row	Own profile still exists after the attempt
SADM-14	API	High	Teacher cannot delete another profile (RLS)	Teacher + victim fixture	DELETE victim's profile with teacher token	Victim's profile still exists
SADM-15	API	High	School cannot delete another profile (RLS)	School + victim fixture	DELETE victim's profile with school token	Victim's profile still exists

SADM-16	UI	High	Role change via Admin panel dropdown	Superadmin + target fixture	Open Admin tab, change target's role dropdown to "school"	Target's DB profile role becomes school within 8s (polled)
SADM-17	API SKIPPED	Med	Superadmin calling manage-teacher → 403	Superadmin + a real teacher created via a school	POST edge function manage-teacher as superadmin	HTTP 403 — function is School-role only
SADM-18	API SKIPPED	Med	Superadmin calling create-teacher → 403	Superadmin fixture	POST edge function create-teacher as superadmin	HTTP 403
SADM-19	—	—	Placeholder: feature-gated block skipped	—	test.skip when FEATURE_SCHOOL_TEACHERS is unset	Explicitly skipped with a documented reason, not silently omitted

9. School → Teachers Management Gated behind FEATURE_SCHOOL_TEACHERS

All 15 cases below (SCHT-01–15) execute only when `FEATURE_SCHOOL_TEACHERS=1` is set (post-deploy of the DB migration + create-teacher / manage-teacher edge functions — this is now set in CI). With the flag unset, a single placeholder test runs `test.skip` instead, bringing this module's file total to 16.

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
SCHT-01	API	High	School can create a pending teacher	School fixture	POST edge function create-teacher with email/name	200, ok:true; new profile has role=teacher, status=pending, correct school_id
SCHT-02	API	High	Teacher account cannot create teachers	Teacher fixture	POST create-teacher as a teacher	HTTP 403
SCHT-03	API	High	School cannot manage another school's teacher	Two school fixtures (A, B) + A's teacher	School B calls manage-teacher (remove) on A's teacher	HTTP 403; teacher profile still exists
SCHT-04	API	High	Deactivate then reactivate toggles status	School + its teacher	1. manage-teacher deactivate 2. reactivate	Status becomes inactive then active
SCHT-05	UI	Med	School sees Employees tab, not global Admin	School fixture	Log in as school, inspect tabs	"Employees" tab visible; "Admin" tab absent
SCHT-06	UI	High	Pending teacher cannot log in	Teacher fixture set to status=pending	Attempt login via UI	Message containing "pending" shown; sign-in form remains
SCHT-07	API	Med	School can remove their own teacher	School + its teacher	POST manage-teacher action=remove	HTTP 200
SCHT-08	API	Low	create-teacher with missing email → 4xx	School fixture	POST create-teacher without email	HTTP 400–499
SCHT-09	API	Low	manage-teacher with unknown action → 400	School + its teacher	POST manage-teacher with action:"fly-to-moon"	HTTP 400–499

SCHT-10	API	Med	Pending teacher self-activates via RPC	Teacher fixture, status=pending	POST RPC activate_my_account with teacher's own token	2xx; profile status becomes active
SCHT-11	API	High	School sees only their own teachers (isolation)	Two schools, each with one teacher	GET /rest/v1/profiles as School A	Contains A's teacher's email; does not contain B's
SCHT-12	API	High	Deactivating a teacher blocks subsequent logins	School + linked teacher	1. Confirm teacher can get a token 2. School deactivates them 3. Retry token grant	Second token request rejected (throws)
SCHT-13	API	Low	create-teacher with duplicate email → error	School fixture + a pre-existing user with the target email	POST create-teacher with an already-registered email	HTTP 400–599
SCHT-14	API	High	Removing a teacher cascades to delete their classes and records	School + linked teacher; a class + record created as that teacher; a superadmin fixture to read rows after the teacher's token is gone	1. Confirm the class + record exist (via superadmin token) 2. POST manage-teacher action=remove 3. Re-check both via superadmin token	Teacher profile gone; the class and its record are both gone (0 rows) — proves the on delete cascade chain, not just the profile deletion
SCHT-15	UI	Med	Remove-teacher confirm dialog discloses class/record deletion	School + linked teacher, logged in as school on the Employees tab	Click "Remove" on the teacher's row; intercept the native confirm() dialog and dismiss it	Dialog message contains "classes", "records", and "cannot be undone"; teacher still exists afterward (dismissed, not confirmed)

10. Classes & Records — Data Layer (REST)

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
CLS-01	API	High	Teacher can create a class	Teacher fixture	POST /rest/v1/classes	HTTP 201; returned row matches submitted id/name
CLS-02	API	Med	Teacher can list their own classes	Teacher fixture + a class	GET classes filtered by id	1 row, correct name
CLS-03	API	Med	Teacher can rename a class	Teacher fixture + a class	PATCH name	HTTP 200; new name returned
CLS-04	API	Med	Teacher can delete a class	Teacher fixture + a class	DELETE the class, then GET it	DELETE returns 204; subsequent GET returns 0 rows
CLS-05	API	Med	Teacher can add students to a class	Teacher fixture + empty class	PATCH students to a 3-name array	Returned students array matches exactly
CLS-06	API	Med	Teacher can remove a student	Class with 3 students	PATCH students omitting one name	Removed name absent; the other two remain
CLS-07	API	High	Teacher cannot read another teacher's class (RLS)	Two teacher fixtures, one class owned by A	GET the class with Teacher B's token	0 rows returned

CLS-08	API	High	Teacher cannot forge another teacher's owner_id on insert (RLS)	Two teacher fixtures (A, B)	Teacher B attempts POST /rest/v1/classes with owner_id set to Teacher A's id	Insert is rejected by RLS (no row created under A's ownership); confirms the WITH CHECK clause on the classes INSERT policy, not just the SELECT/DELETE policies already covered by CLS-07/09
CLS-09	API	High	Teacher cannot delete another's class (RLS)	Two teacher fixtures, one class owned by A	DELETE the class with Teacher B's token, then re-check as A	Class still visible to A afterward
CLS-10	API	Med	Teacher can create records for own class	Teacher fixture + a fresh class	POST /rest/v1/records	HTTP 201; class_id matches
CLS-11	API	Med	Teacher can read their own records	Class + record fixture	GET records filtered by class_id	1 row returned
CLS-12	API	Med	Teacher can upsert updated record data	Existing record	POST with resolution=merge-duplicates and new data	HTTP 200/201; updated payment/attendance values reflected
CLS-13	API	High	Teacher cannot read another's records (RLS)	Two teacher fixtures, record owned by A	GET the record with Teacher B's token	0 rows returned
CLS-14	API	High	Deleting a class cascades to its records	Class + record fixture	1. Confirm record exists 2. DELETE the class 3. Re-check the record	Record row is gone after the class is deleted
CLS-15	API	Med	Superadmin can read any teacher's class	Superadmin + teacher-owned class	GET the class with admin token	1 row returned, correct id
CLS-16	API	Med	Superadmin can read any teacher's records	Superadmin + teacher-owned record	GET the record with admin token	1 row returned, correct class_id

11. Profile Self-Management

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
PROF-01	API	Med	Teacher can update own first_name	Teacher fixture	PATCH own profile's first_name	HTTP 200; new value returned
PROF-02	API	Med	Teacher can update own last_name	Teacher fixture	PATCH own profile's last_name	HTTP 200; new value returned
PROF-03	API	High	Teacher cannot change own role	Teacher fixture	PATCH own role to superadmin	Role remains teacher (trigger blocks it)
PROF-04	API	High	Teacher cannot update another's profile	Two teacher fixtures	PATCH peer's first_name with own token	0 rows affected; peer's name unchanged
PROF-05	API	Med	Superadmin can update any user's name	Superadmin + target fixture	PATCH target's first_name with admin token	HTTP 200; new value returned

12. Records Tab — UI

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
RECU-01	UI	Med	Empty state when teacher has no	Teacher fixture, no classes	Open Records tab	Empty-state message containing "no classes for

			classes			records"
RECUI-02	UI	High	Selecting a class lists its students	Class with 2 students	Select the class in the Records dropdown	Table shows exactly 2 rows, both student names visible
RECUI-03	UI	High	Marking present updates row + summary	Class with 2 students	Click the "Absent" toggle for one student	Toggle flips to "Present"; "Present today" summary reads 1 / 2
RECUI-04	UI	Med	"All present" marks every student	Class with 2 students	Click "✓ All present"	Both rows show "Present"; summary reads 2 / 2
RECUI-05	UI	High	Grade set on a student persists	Class with 2 students	1. Select grade "5" for a student 2. Wait for debounced sync 3. Reload and re-select the class	Grade dropdown still reads "5" after reload
RECUI-06	UI	High	Payment toggle updates summary + persists	Class with 2 students	1. Click "Not paid" toggle 2. Verify "Paid" summary = 1/2 3. Wait, reload, re-select class	Toggle still reads "Paid" after reload
RECUI-07	UI	Med	Student profile shows stats and saves a note	Class with 1 student, marked present once	1. Click the student's name 2. Check attendance stat 3. Fill the notes textarea 4. Wait, reload, reopen profile	Modal shows correct name and "1" attendance; note text persists after reload
RECUI-08	UI	High	Failed record save is logged and not silently persisted	Class with 1 student	1. Intercept the records upsert request, fulfill a simulated 500 2. Toggle attendance 3. Wait for the console error 4. Remove the interception, reload	Console logs "Failed to save record"; after reload the attendance mark has reverted — the write never actually landed despite the optimistic UI update
RECUI-09	UI	Med	A note containing HTML markup is stored and redisplayed as literal text	Class with 1 student	1. Open the student profile 2. Fill the notes textarea with a payload containing a <code><script></code> tag 3. Wait, reload, reopen the profile	Notes textarea value round-trips verbatim; no script executes

13. PDF Preview Modal

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
PDF-01	UI	High	Generating a worksheet opens the modal with correct info	Teacher fixture, logged in	1. Select topic "am / is / are" 2. Click "Generate worksheet"	Modal visible; title = topic name; subtitle contains "Grade 2" and "exercises"
PDF-02	UI	High	Answer-key toggle shows/hides answers	Modal open with a fill-in worksheet	Click the answer-key switch, then click again	Toggle gains/loses active class; blank answers and the Answer Key block appear then disappear
PDF-03	UI	Med	"New set" regenerates without closing the modal	Modal open	Click "🔄 New set"	Modal remains visible; item count unchanged
PDF-04	UI	High	Close button hides the modal	Modal open	Click the "X" button	Modal removed from the DOM
PDF-05	UI	High	Escape key hides the modal	Modal open	Press Escape	Modal removed from the DOM

14. Visual Regression added v1.4

Pixel-diff snapshots of four key screens, using Playwright's `toHaveScreenshot()`. Baselines live under `tests/visual/visual-regression.spec.ts-snapshots/` and must only be (re)generated on CI's `ubuntu-latest` runner via the "Update Visual Regression Baselines" workflow — see Test Plan §6. Fixed viewport 1280×900, `maxDiffPixelRatio: 0.02`, run serially.

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
VIS-01	UI	High	Login screen renders consistently	No session	1. Visit / 2. Screenshot the <code>.auth-card</code> element	Matches the checked-in <code>login.png</code> baseline within 2% diff-pixel ratio
VIS-02	UI	Med	Dashboard (generator tab, no topic selected) renders consistently	Fresh confirmed teacher fixture, logged in	Screenshot the <code>.app</code> element right after login	Matches the checked-in <code>dashboard.png</code> baseline within 2% diff-pixel ratio
VIS-03	UI	Med	Worksheet generator settings (topic selected) renders consistently	Fresh teacher fixture, logged in	1. Click the "am / is / are" topic card 2. Screenshot the <code>.app</code> element	Matches the checked-in <code>worksheet-generator.png</code> baseline within 2% diff-pixel ratio
VIS-04	UI	High	PDF preview modal renders consistently	Fresh teacher fixture, logged in	1. Select "am / is / are" 2. Click "Generate worksheet" 3. Screenshot the <code>.pdf-modal-card</code> element	Matches the checked-in <code>pdf-modal.png</code> baseline within 2% diff-pixel ratio

15. Classes Tab — UI added v1.5

UI click-through coverage for the Classes tab (create/add/remove/delete had only REST coverage in Module 10 before this) plus XSS/HTML-injection resilience in its two free-text fields. `tests/app/classes-ui.spec.ts`.

ID	Type	Pri.	Title	Preconditions	Steps	Expected Result
CLSUI-01	UI	High	Creating a class via the "add class" form adds it to the list	Teacher fixture, logged in	1. Fill the class-name input 2. Click "+ Add class"	A <code>.class-card</code> for the new class appears, showing "0 students"
CLSUI-02	UI	High	Adding a student via the roster input lists them on the class card	Teacher fixture + an empty class	1. Fill the "Student name" input on the class card 2. Click "+ Add"	A student tag with the name appears; the count reads "1 student"
CLSUI-03	UI	High	Removing a student via its roster tag drops them from the class card	Class with 2 students	Click the "×" (title="Remove") button on one student's tag	That tag disappears; the other student's tag remains; count updates to "1 student"
CLSUI-04	UI	High	Deleting a class via its delete button removes it from the list	Teacher fixture + a class	Click "Delete class" on the class card	The <code>.class-card</code> is no longer present
CLSUI-05	UI	Med	A class name containing a script tag renders as literal text, not markup	Teacher fixture, logged in	Fill the class-name input with a <code><script></code> payload and submit	The card shows the literal string; no <code><script></code> element exists inside <code>.class-name</code> ; the payload never executes
CLSUI-06	UI	Med	A student name containing an HTML tag renders as literal text, not markup	Teacher fixture + an empty class	Fill the "Student name" input with an <code></code> payload and submit	The tag shows the literal string; no <code></code> element exists inside the tag; the payload never executes